

🔗 See Topic 14 → Dynamic Programming Masterclass

🎯 **GOAL:** Master Greedy Algorithms! Learn the Greedy Choice Property, Exchange Argument Proofs, Counter-Example Analysis, Interval Sorting, Farthest Reach Jump Windows, Circuit Deficit tracking, and solve all NeetCode Greedy questions!

📌 REUSABLE GREEDY UTILITIES

```
// Interval sorting by end time (Canonical Greedy Choice)
Arrays.sort(intervals, (a, b) → Integer.compare(a[1], b[1]));

// Max Reach Tracker Pattern (Jump Game / Gas Station)
int maxReach = 0;
for (int i = 0; i < nums.length; i++) {
    if (i > maxReach) return false; // unreachable gap
    maxReach = Math.max(maxReach, i + nums[i]);
}
```

- 💡 **AHA MOMENT #1: GREEDY ALGORITHMS NEVER BACKTRACK!**
- A Greedy algorithm makes the ****locally optimal choice**** at each step without ever re-evaluating or undoing past decisions. It works **ONLY** when:
- Greedy Choice Property:** A globally optimal solution can be arrived at by making locally optimal (greedy) choices!
 - Optimal Substructure:** An optimal solution to the problem contains optimal solutions to its subproblems!

1 CORE GREEDY TERMINOLOGY

- **Local Optimum:** Best immediate choice available at step 'i'.
- **Global Optimum:** Absolutely best overall solution to the entire problem.
- **No Backtracking:** Decisions are final (unlike Backtracking or DP).
- **Sorting Prerequisite:** Most greedy array/interval problems require sorting input first!

2 GREEDY VS DP VS BACKTRACKING MATRIX

Feature	Greedy	Dynamic Programming	Backtracking
Decisions Made	1 Choice per step (Final)	Explores ALL subproblems & picks min/max	Explores ALL branches with state undo
Time Complexity	✅ $O(N)$ or $O(N \log N)$ (Fastest!)	$O(N^2)$ or $O(N \cdot W)$	$O(2^N)$ or $O(N!)$
Correctness Proof	Requires Exchange Proof	Proven by Recurrence Base Cases	Guaranteed by Exhaustive Search

📌 **KEY TAKEAWAY:** Reach for Greedy when a problem asks for "Min/Max Activities, Refuels, or Intervals" AND you can prove local choice leads to global optimum!



💡 AHA MOMENT #2: WHY GREEDY FAILS (COUNTER-EXAMPLE ANALYSIS)

Greedy fails whenever a local greedy choice **destroys future possibilities**!

• **Example: Coin Change for Amount = 6 with Coins [1, 3, 4]:**

- *Greedy Choice:* Pick largest coin '4' → Remaining '2' → Pick '1' + '1' → Total 3 coins ('4, 1, 1').

- *Optimal Solution (DP):* Pick '3' + '3' → Total 2 coins! ❌ **Greedy Fails!**

1 THE EXCHANGE ARGUMENT PROOF TECHNIQUE

PROOF STEPS:

1. Assume there exists an Optimal Solution \$O\$.
2. Compare \$O\$ with Greedy Solution \$G\$.
3. Show that swapping any choice in \$O\$ with the Greedy choice in \$G\$ does NOT decrease solution quality!
4. Conclude that \$G\$ is as optimal as \$O\$!

2 4-STEP GREEDY WORKFLOW

1. **Sort Input:** Sort by End Time, Start Time, or Value Ratio.
2. **Identify Local Choice:** Pick element that maximizes immediate gain.
3. **Check Counter-Examples:** Can local choice ruin future states?
4. **Implement Single Pass:** Iterate array in \$O(N)\$ time!



FAANG INTERVIEW WARNING: Never present a Greedy solution without explaining WHY the greedy choice is optimal! Always mention a counter-example where Greedy would fail!

**💡 AHA MOMENT #3: SORT BY END TIME VS SORT BY START TIME**

- **Sort by End Time** (`'b[1] - a[1]'`): For *Interval Scheduling / Non-Overlapping Intervals*! Picking intervals that finish earliest leaves maximum room for future intervals!
- **Sort by Start Time** (`'a[0] - b[0]'`): For *Merge Intervals*! Processing intervals in order of appearance allows merging overlapping bounds sequentially!

1 SORT BY END TIME (MAX ACTIVITIES / NON-OVERLAP)

```
Arrays.sort(intervals, (a, b) → Integer.compare(a[1], b[1]));
int count = 0, prevEnd = Integer.MIN_VALUE;
for (int[] inv : intervals) {
    if (inv[0] ≥ prevEnd) {
        count++; prevEnd = inv[1]; // Pick earliest ending interval!
    }
}
```

2 SORT BY START TIME (MERGE INTERVALS)

```
Arrays.sort(intervals, (a, b) → Integer.compare(a[0], b[0]));
List<int[]> merged = new ArrayList<>();
for (int[] inv : intervals) {
    if (merged.isEmpty() || merged.get(merged.size() - 1)[1] < inv[0])
    {
        merged.add(inv);
    } else {
        merged.get(merged.size() - 1)[1] =
        Math.max(merged.get(merged.size() - 1)[1], inv[1]);
    }
}
```

🚩 **KEY TAKEAWAY:** Always use `Integer.compare(a, b)` instead of `a - b` to prevent integer overflow bugs!

💡 **AHA MOMENT #4: FARTHEST REACH IMPLICIT BFS WINDOW**

Instead of trying all jump combinations (Backtracking $O(2^N)$), maintain a greedy 'farthest' index reach!
 In **Jump Game II**, treat `[currStart, currEnd]` as a BFS level! Increment 'jumps++' ONLY when index 'i' reaches 'currEnd'!

1 JUMP GAME I (CAN REACH END?)

```
public boolean canJump(int[] nums) {
    int farthest = 0;
    for (int i = 0; i < nums.length; i++) {
        if (i > farthest) return false; // Trapped!
        farthest = Math.max(farthest, i + nums[i]);
    }
    return true;
}
```

2 JUMP GAME II (MINIMUM JUMPS)

```
public int jump(int[] nums) {
    int jumps = 0, currEnd = 0, farthest = 0;
    for (int i = 0; i < nums.length - 1; i++) {
        farthest = Math.max(farthest, i + nums[i]);
        if (i == currEnd) { // Reached end of current jump level
            jumps++;
            currEnd = farthest;
        }
    }
    return jumps;
}
```

🚀 **KEY TAKEAWAY:** Jump Game II runs in $O(N)$ time and $O(1)$ space using implicit BFS level windows!

**💡 AHA MOMENT #5: CIRCUIT DEFICIT & RESET GREEDY LOGIC (GAS STATION)****In Gas Station (LC 134):**

1. If `totalGas < totalCost`, a complete circular trip is IMPOSSIBLE!
2. If `currGas < 0` at station `i`, NO station from `start` to `i` can be the starting point! Reset `start = i + 1` and `currGas = 0`!

🔪 GAS STATION (LC 134) \$O(N)\$ SINGLE-PASS IMPLEMENTATION

```
public int canCompleteCircuit(int[] gas, int[] cost) {
    int totalGas = 0, totalCost = 0, currGas = 0, start = 0;
    for (int i = 0; i < gas.length; i++) {
        totalGas += gas[i];
        totalCost += cost[i];
        currGas += gas[i] - cost[i];
        if (currGas < 0) { // Deficit! Reset start to next station
            start = i + 1;
            currGas = 0;
        }
    }
    return totalGas ≥ totalCost ? start : -1;
}
```

🔑 KEY TAKEAWAY: Checking `totalGas >= totalCost` guarantees that the reset `start` station will successfully complete the full circuit!

**💡 AHA MOMENT #6: MAX FREQUENCY DRIVES IDLE SLOTS (TASK SCHEDULER)**

In **Task Scheduler (LC 621)**, the task with the highest frequency ('maxFreq') creates the bottleneck frame layout!

Formula: $(\text{maxFreq} - 1) * (n + 1) + \text{countMaxFreq}$

🔪 TASK SCHEDULER (LC 621) \$O(N)\$ GREEDY FORMULA

```
public int leastInterval(char[] tasks, int n) {
    int[] freq = new int[26];
    int maxFreq = 0, countMaxFreq = 0;
    for (char c : tasks) {
        freq[c - 'A']++;
        maxFreq = Math.max(maxFreq, freq[c - 'A']);
    }
    for (int f : freq) if (f == maxFreq) countMaxFreq++;
    int ans = (maxFreq - 1) * (n + 1) + countMaxFreq;
    return Math.max(ans, tasks.length);
}
```

🔑 **KEY TAKEAWAY:** ' $\text{Math.max}(\text{ans}, \text{tasks.length})$ ' handles cases where there are enough unique tasks that zero idle slots are needed!

**💡 AHA MOMENT #7: LAST OCCURRENCE MAP & INSERTION RECONSTRUCTION**

- **Partition Labels (LC 763):** Track the last occurrence of each character. Expand partition boundary `maxEnd = Math.max(maxEnd, lastIndex[c])`!
- **Queue Reconstruction by Height (LC 406):** Sort by height **descending**, `k` **ascending**. Taller people are already placed, so inserting person at index `k` maintains valid counts!

1 PARTITION LABELS (LC 763)

```
public List<Integer> partitionLabels(String s) {
    int[] last = new int[26];
    for (int i = 0; i < s.length(); i++) last[s.charAt(i) - 'a'] = i;
    List<Integer> res = new ArrayList<>();
    int start = 0, maxEnd = 0;
    for (int i = 0; i < s.length(); i++) {
        maxEnd = Math.max(maxEnd, last[s.charAt(i) - 'a']);
        if (i == maxEnd) {
            res.add(i - start + 1); start = i + 1;
        }
    }
    return res;
}
```

2 QUEUE RECONSTRUCTION (LC 406)

```
public int[][] reconstructQueue(int[][] people) {
    Arrays.sort(people, (a, b) -> a[0] != b[0] ? b[0] - a[0] : a[1] - b[1]);
    List<int[]> res = new ArrayList<>();
    for (int[] p : people) res.add(p[1], p); // Insert at index k!
    return res.toArray(new int[people.length][]);
}
```

🚀 **KEY TAKEAWAY:** Queue Reconstruction uses `res.add(p[1], p)` to greedily place items in list order!

 **GREEDY INTERVIEW DECISION TREE (MERMAID)**

 **PRINTABLE GREEDY CHEAT SHEET**

Pattern Type	Target Problem	Greedy Choice Strategy	Complexity
Interval Non-Overlap	Non-overlapping Intervals (LC 435)	Sort by End Time <code>b[1] - a[1]</code>	$O(N \log N)$ Time, $O(1)$ Space
Merge Intervals	Merge Intervals (LC 56)	Sort by Start Time <code>a[0] - b[0]</code>	$O(N \log N)$ Time, $O(N)$ Space
Farthest Reach	Jump Game II (LC 45)	Implicit BFS level <code>currEnd = farthest</code>	$O(N)$ Time, $O(1)$ Space
Circuit Deficit	Gas Station (LC 134)	Reset <code>start = i + 1</code> on deficit	$O(N)$ Time, $O(1)$ Space
Frequency Scheduling	Task Scheduler (LC 621)	Max freq bottleneck formula	$O(N)$ Time, $O(1)$ Space
String Partition	Partition Labels (LC 763)	Last index map tracking <code>maxEnd</code>	$O(N)$ Time, $O(1)$ Space

 **FAANG COACH TIP:** Print this page and review it 10 minutes before your technical interview!



1. INTERVAL SCHEDULING TEMPLATE (SORT END TIME)

```
int intervalScheduling(int[][] intervals) {
    Arrays.sort(intervals, (a, b) → Integer.compare(a[1], b[1]));
    int count = 0, prevEnd = Integer.MIN_VALUE;
    for (int[] inv : intervals) {
        if (inv[0] ≥ prevEnd) {
            count++; prevEnd = inv[1];
        }
    }
    return count;
}
```

2. FARTHEST REACH JUMP TEMPLATE

```
int jumpGame(int[] nums) {
    int jumps = 0, currEnd = 0, farthest = 0;
    for (int i = 0; i < nums.length - 1; i++) {
        farthest = Math.max(farthest, i + nums[i]);
        if (i == currEnd) {
            jumps++; currEnd = farthest;
        }
    }
    return jumps;
}
```

🔪 **REUSABLE CODE:** Copy-paste these templates directly into your interview whiteboard or IDE!

3. CIRCUIT DEFICIT RESET TEMPLATE

```
int circuitDeficit(int[] gain, int[] cost) {
    int total = 0, curr = 0, start = 0;
    for (int i = 0; i < gain.length; i++) {
        int diff = gain[i] - cost[i];
        total += diff; curr += diff;
        if (curr < 0) { start = i + 1; curr = 0; }
    }
    return total ≥ 0 ? start : -1;
}
```

4. TWO-PASS ARRAY GREEDY TEMPLATE (CANDY)

```
int twoPassGreedy(int[] ratings) {
    int n = ratings.length; int[] arr = new int[n];
    Arrays.fill(arr, 1);
    for (int i = 1; i < n; i++) if (ratings[i] > ratings[i-1]) arr[i]
    = arr[i-1] + 1;
    for (int i = n - 2; i ≥ 0; i--) if (ratings[i] > ratings[i+1])
    arr[i] = Math.max(arr[i], arr[i+1] + 1);
    int sum = 0; for (int x : arr) sum += x; return sum;
}
```

🚀 **KEY TAKEAWAY:** Memorize these core templates to solve 95% of FAANG Greedy problems!

1. LEETCODE 455 — ASSIGN COOKIES **Easy ★★★★★** Sort + Two Pointers [Amazon](#)**Problem:**

Maximize content children using cookies of size `s[j]` for greedy greed factor `g[i]`.

```
public int findContentChildren(int[] g, int[] s) {
    Arrays.sort(g); Arrays.sort(s);
    int i = 0, j = 0;
    while (i < g.length && j < s.length) {
        if (s[j] ≥ g[i]) i++; // Cookie satisfies child!
        j++; // Try next cookie
    }
    return i;
}
```

2. LEETCODE 860 — LEMONADE CHANGE **Easy ★★★★★** Bill Change Priority [Amazon](#)**Problem:**

Provide correct change (\$5, \$10, \$20) for lemonade customers.

```
public boolean lemonadeChange(int[] bills) {
    int five = 0, ten = 0;
    for (int b : bills) {
        if (b == 5) five++;
        else if (b == 10) {
            if (five == 0) return false;
            five--; ten++;
        } else { // b == 20
            if (ten > 0 && five > 0) { ten--; five--; } // Greedy: Use
$10 bill first!
            else if (five ≥ 3) five -= 3;
            else return false;
        }
    }
    return true;
}
```

 **AHA MOMENT (LC 860):** When giving \$15 change for a \$20 bill, greedily use a \$10 bill first because \$5 bills are more versatile!



3. LEETCODE 55 — JUMP GAME I

Medium ★★★★★ Farthest Reach Track Amazon Meta

Problem:

Determine if you can reach the last index from the first index.

```
public boolean canJump(int[] nums) {
    int farthest = 0;
    for (int i = 0; i < nums.length; i++) {
        if (i > farthest) return false;
        farthest = Math.max(farthest, i + nums[i]);
    }
    return true;
}
```

4. LEETCODE 45 — JUMP GAME II

Medium ★★★★★ Implicit BFS Level Window Amazon Google

Problem:

Minimum number of jumps to reach the last index.

```
public int jump(int[] nums) {
    int jumps = 0, currEnd = 0, farthest = 0;
    for (int i = 0; i < nums.length - 1; i++) {
        farthest = Math.max(farthest, i + nums[i]);
        if (i == currEnd) {
            jumps++; currEnd = farthest;
        }
    }
    return jumps;
}
```

💡 **AHA MOMENT (LC 45):** Increment `jumps++` ONLY when index `i` reaches `currEnd`!

5. LEETCODE 134 — GAS STATION

Medium
★★★★★

Circuit Deficit Reset

Amazon

Meta

Problem:

Find starting gas station index to complete a full circular trip.

```
public int canCompleteCircuit(int[] gas, int[] cost) {
    int totalGas = 0, totalCost = 0, currGas = 0, start = 0;
    for (int i = 0; i < gas.length; i++) {
        totalGas += gas[i]; totalCost += cost[i];
        currGas += gas[i] - cost[i];
        if (currGas < 0) { start = i + 1; currGas = 0; }
    }
    return totalGas ≥ totalCost ? start : -1;
}
```

6. LEETCODE 763 — PARTITION LABELS

Medium
★★★★★

Last Index Map Boundary

Amazon

Problem:

Partition string so that each letter appears in at most one part.

```
public List<Integer> partitionLabels(String s) {
    int[] last = new int[26];
    for (int i = 0; i < s.length(); i++) last[s.charAt(i) - 'a'] = i;
    List<Integer> res = new ArrayList<>();
    int start = 0, maxEnd = 0;
    for (int i = 0; i < s.length(); i++) {
        maxEnd = Math.max(maxEnd, last[s.charAt(i) - 'a']);
        if (i == maxEnd) { res.add(i - start + 1); start = i + 1; }
    }
    return res;
}
```

💡 **AHA MOMENT (LC 763):** Updating `maxEnd = Math.max(maxEnd, last[char])` expands partition boundary greedily!`

7. LEETCODE 56 — MERGE INTERVALS


Medium

★★★★★

Sort Start Time Merge

Amazon

Meta

Problem:

Merge all overlapping intervals into non-overlapping result array.

```
public int[][] merge(int[][] intervals) {
    Arrays.sort(intervals, (a, b) → Integer.compare(a[0], b[0]));
    List<int[]> res = new ArrayList<>();
    for (int[] inv : intervals) {
        if (res.isEmpty() || res.get(res.size() - 1)[1] < inv[0])
            res.add(inv);
        else res.get(res.size() - 1)[1] = Math.max(res.get(res.size() - 1)[1], inv[1]);
    }
    return res.toArray(new int[res.size()][]);
}
```

8. LEETCODE 435 — NON-OVERLAPPING INTERVALS


Medium

★★★★★

Sort End Time Removal

Amazon

Problem:

Minimum number of intervals to remove to make remaining non-overlapping.

```
public int eraseOverlapIntervals(int[][] intervals) {
    Arrays.sort(intervals, (a, b) → Integer.compare(a[1], b[1]));
    int removals = 0, prevEnd = Integer.MIN_VALUE;
    for (int[] inv : intervals) {
        if (inv[0] ≥ prevEnd) prevEnd = inv[1]; // Keep interval
        else removals++; // Remove overlapping interval
    }
    return removals;
}
```


KEY TAKEAWAY: Merge Intervals sorts by START time; Non-Overlapping Intervals sorts by END time!

9. LEETCODE 406 — QUEUE RECONSTRUCTION BY HEIGHT

 **Medium ★★★★★** **Sort Descending Height Insert** [Amazon](#) [Google](#)

Problem:

Reconstruct queue of people `[h, k]` where `k` is number of taller people ahead.

```
public int[][] reconstructQueue(int[][] people) {
    Arrays.sort(people, (a, b) → a[0] ≠ b[0] ? b[0] - a[0] : a[1] - b[1]);
    List<int[]> res = new ArrayList<>();
    for (int[] p : people) res.add(p[1], p);
    return res.toArray(new int[people.length][1]);
}
```

10. LEETCODE 135 — CANDY

 **Hard ★★★★★** **Two-Pass Left/Right Greedy** [Amazon](#) [Google](#)

Problem:

Minimum candies needed to distribute to children satisfying rating neighbor rules.

```
public int candy(int[] ratings) {
    int n = ratings.length; int[] candies = new int[n];
    Arrays.fill(candies, 1);
    for (int i = 1; i < n; i++) { // Left to Right Pass
        if (ratings[i] > ratings[i - 1]) candies[i] = candies[i - 1] + 1;
    }
    for (int i = n - 2; i ≥ 0; i--) { // Right to Left Pass
        if (ratings[i] > ratings[i + 1]) candies[i] = Math.max(candies[i], candies[i + 1] + 1);
    }
    int sum = 0; for (int c : candies) sum += c; return sum;
}
```

 **FAANG COACH TIP:** LC 135 Candy requires 2 passes (Left-to-Right + Right-to-Left) with `Math.max()` to satisfy BOTH neighbor constraints!

★ THE 4 GOLDEN LAWS OF GREEDY ALGORITHMS

1. **No Backtracking Law:** Greedy choices are permanent! Prove local choice leads to global optimum.
2. **Interval Sorting Rule:** Non-overlapping intervals sort by END time; Merging intervals sort by START time.
3. **Farthest Reach Rule:** Jump Game maintains `farthest = max(farthest, i + nums[i])`.
4. **Circuit Deficit Reset:** Reset `start = i + 1` on negative gas balance!

🚫 WHEN GREEDY FAILS (DO NOT USE!)

- **Coin Change with Non-Canonical Denominations:** `[1, 3, 4]` for amount `6` → Use ****Dynamic Programming****!
- **0/1 Knapsack with Weight Constraints:** Fractional choices not allowed → Use ****Dynamic Programming****!

📁 NEETCODE & BLIND 75 CONFIDENCE TRACKER

🟢 Easy	LC 455 Assign Cookies	🟢 Easy	LC 860 Lemonade Change	🟡 Med	LC 55 Jump Game I
🟡 Med	LC 45 Jump Game II	🟡 Med	LC 134 Gas Station	🟡 Med	LC 763 Partition Labels
🟡 Med	LC 56 Merge Intervals	🟡 Med	LC 435 Non-overlapping	🟡 Med	LC 406 Queue Recon
🔴 Hard	LC 135 Candy				

👉 Next Master Topic: Topic 16 → Intervals Masterclass

🇬🇧 TOPIC 15 GREEDY ALGORITHMS MASTERCLASS COMPLETE! Turn the page to **Topic 16: Intervals Masterclass!**